Network Time Synchronization System: Project Guide

Course: Computer Networks, Jackfruit Mini Project (SDN + Socket Programming) **Topic No.:** 23 **Protocol:** UDP
Team size: 3

1. Project in One Line

Build a simplified **NTP-style time synchronization application** using **UDP sockets**, where multiple clients (each with a deliberately wrong clock) synchronize with a reference server, running inside a **Mininet** network controlled by an **SDN controller** that prioritizes synchronization traffic.

2. Basics You Need First

2.1 What is a socket?

A socket is an endpoint for sending and receiving data over a network. A program creates a socket, binds it to an IP address and port, and then sends or receives messages.

2.2 TCP vs UDP

	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Guaranteed, ordered delivery	No guarantee, packets may be lost or reordered
Speed / overhead	Slower, more overhead	Faster, lightweight
Use cases	Web, file transfer	Time sync, streaming, DNS, games

Time synchronization uses **UDP** because it is fast and predictable. TCP's retransmissions and buffering would add variable delay and ruin timing accuracy.

2.3 What is clock drift?

Every computer has its own hardware clock. Over time these clocks run slightly faster or slower, so two computers drift apart. Clock synchronization protocols (such as NTP) correct this.

2.4 What is SDN?

Software Defined Networking separates the network into two parts:

- **Data plane:** switches that forward packets.
- **Control plane:** a central **controller** (software) that decides how switches should forward packets, by installing **flow rules** in them via **OpenFlow**.

This lets us change network behavior (priorities, routes, blocking) dynamically from code.

2.4.1 Tools you will use

- **Mininet:** emulates hosts, switches and links on one machine.
- **Open vSwitch (OVS):** the virtual switch used by Mininet.
- **Ryu (or POX):** the Python SDN controller.
- **iperf / custom UDP generator:** creates background congestion.
- **Python `socket`, `threading`:** the application itself.
- **matplotlib / pandas:** graphs and result analysis.

3. The Core Idea: How Time Synchronization Works

Asking the server "what time is it?" does not work, because the reply takes time to travel, so the time is already outdated when it arrives. Instead we exchange **four timestamps**:

Timestamp	Meaning
T1	Client clock when the request is **sent**
T2	Server clock when the request is **received**
T3	Server clock when the reply is **sent**
T4	Client clock when the reply is **received**

```
Client                          Server
  |--- request (T1) ----------->|  T2 (arrives)
  |                             |  ...processing...
  |<-------- reply (T3) --------|  T3 (leaves)
  T4 (arrives)
```

Formulas

Round-trip delay

```
D = (T4 - T1) - (T3 - T2)
```

Total time on the client minus the time the server spent processing.

Clock offset

```
θ = ((T2 - T1) + (T3 - T4)) / 2
```

Correction

```
Corrected time = client clock + θ
```

Worked example (milliseconds)

T1 = 100, T2 = 160, T3 = 161, T4 = 111

- D = (111 – 100) – (161 – 160) = **10 ms**
- θ = ((160 – 100) + (161 – 111)) / 2 = (60 + 50) / 2 = **55 ms**
- The client is 55 ms behind, so it adds 55 ms to its clock.

Key limitation: asymmetric delay

The formula assumes the forward delay and the return delay are **equal**. If they are not, the offset error is:

```
error = (d_fwd - d_rev) / 2
```

This is why congestion, which affects the two directions unevenly, hurts accuracy. It is the main point of your experiments.

4. Project Requirements Explained

#	Requirement	What you actually build
1	UDP timestamp exchange	A client and server exchanging a UDP packet carrying T1, T2, T3 (the client records T4)
2	Round-trip delay	Compute D from the formula after receiving the reply
3	Clock offset	Compute θ and apply it to the client clock
4	Multiple clients	Each client is a **thread** with its own socket and its own clock skew
5	Prioritize sync traffic	Set `IP_TOS` (DSCP) on sync packets; use a priority queue on the server; the SDN controller gives these packets a high-priority queue
6	Controlled network delay	Artificial delay on the server or proxy, and Mininet link parameters (`delay=`, `loss=`)
7	Measure sync error	Compare the client's corrected time with the true server time
8	Compare under congestion	Run background load; compare sync error and RTT jitter at low vs high traffic, with priority ON vs OFF

Important implementation notes

- **Simulating a wrong clock:** all Mininet hosts share your machine's real clock. So each client uses a software clock: `client_time = time.time() + skew`. Because the real clock is shared, you know the **true time**, which makes the error easy to measure.
- **Priority marking:** DSCP EF = 46 → TOS byte = `46 << 2 = 184 = 0xB8`.

  ```
  sock.setsockopt(socket.IPPROTO_IP, socket.IP_TOS, 0xB8)
  ```

- **Sync error:** `error = |corrected_client_time - true_time|`.

5. What the Rubric Adds (Mininet + SDN)

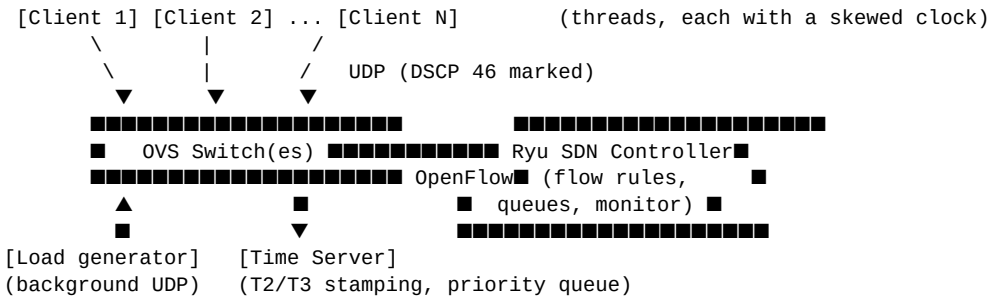
The grading document requires more than the socket app:

- The app must run in a **Mininet** topology.
- An **SDN controller** must be functional with proper **flow rules**.
- The socket app and the SDN network must **interact** (application or network events influence SDN behavior).
- **Dynamic SDN behavior** is required (for example prioritization, rerouting, monitoring, blocking or recovery).
- At least one **challenging scenario** (congestion, packet loss, link failure, server overload).
- **Performance comparison** against a baseline (static or no-priority approach).
- **GitHub repo** with topology, code, config, test cases, reproduction steps and full documentation.
- **Viva:** each student must show individual understanding.

Marks breakdown (40 total)

Deliverable	Component	Marks
D1 (15)	Problem understanding & architecture	2
	TCP/UDP socket implementation	4
	Mininet + SDN implementation	4
	Socket–SDN integration	3
	Initial testing & demo	2
D2 (25)	Complete application functionality	4
	Dynamic SDN functionality	6
	Challenging scenario / robustness	4
	Performance evaluation & comparison	5
	Documentation, final demo & viva	6

6. System Architecture



Communication flow

A client sends a UDP sync request marked with high priority.

The switch matches it against controller-installed rules and puts it in the high-priority queue.

The server stamps T2 and T3 and replies.

The client records T4, computes D and θ, and corrects its clock.

Meanwhile the load generator creates congestion; the controller monitors port statistics and reacts dynamically.

Results are logged and analyzed.

Dynamic SDN idea: the controller polls port statistics; when utilization crosses a threshold (congestion), it installs or updates rules so that sync traffic keeps priority (and background traffic is rate-limited or rerouted).

7. Experiments to Run

Experiment	Traffic	Priority	Metrics
E1 (baseline)	Low	OFF	RTT, jitter, sync error
E2	Low	ON	RTT, jitter, sync error
E3	High (congested)	OFF	RTT, jitter, sync error

Experiment	Traffic	Priority	Metrics
E4	High (congested)	ON (static priority)	RTT, jitter, sync error
E5	High (congested)	ON with **dynamic SDN** control	RTT, jitter, sync error
E6 (challenge)	Congestion + packet loss or link failure	Dynamic SDN	Recovery time, error

Run each experiment with multiple clients (for example 3, 5, 10) and repeat several times. Present average, min, max and standard deviation, with graphs.

8. Repository Structure (suggested)

```
network-time-sync/
■■■ README.md
■■■ src/
■   ■■■ server.py          # UDP time server
■   ■■■ client.py          # client threads + clock skew
■   ■■■ protocol.py        # packet format, shared helpers
■   ■■■ loadgen.py         # background traffic generator
■■■ topology/
■   ■■■ topo.py            # Mininet topology
■■■ controller/
■   ■■■ sync_controller.py # Ryu app
■■■ experiments/
■   ■■■ run_experiments.sh
■   ■■■ analyze.py         # graphs and statistics
■■■ results/               # CSVs and graphs
■■■ docs/
    ■■■ architecture.md
    ■■■ test_cases.md
    ■■■ report.md
```

9. Work Division (3 people, equal workload)

The work is split into **three balanced roles**. Each role has a **core component** (the part the person owns and explains in the viva) plus a share of the **cross-cutting work** (testing, experiments, documentation), so the total effort is equal.

Role 1: Socket Application Owner

Core component: UDP time-sync application

Task	Details
Packet format	Define the T1–T4 message structure (shared with the team)
UDP server	Receive requests, stamp T2/T3, reply; threading for concurrent clients; priority queue
UDP client	T1/T4 recording, D and θ calculation, clock correction
Multi-client	Client threads, each with its own socket and clock skew
Error handling	Timeouts, lost packets, malformed packets
Logging	Write RTT, offset and error to CSV
Shared share	Documentation of the socket layer; experiments **E1 and E2** and analysis of their results

Role 2: Mininet + SDN Owner

Core component: network topology and controller

Task	Details
Environment setup	Mininet, Open vSwitch and Ryu installed and working
Topology	Hosts, switches and links with configurable `delay` and `loss`
Basic controller	Learning switch and baseline forwarding
Priority rules	Match DSCP 46, assign to high-priority OVS queue
Dynamic SDN	Port statistics monitoring, congestion detection, rule updates
Link failure handling	Rerouting or recovery logic (for the challenging scenario)
Shared share	Documentation of the SDN layer; experiments **E3 and E4** and analysis of their results

Role 3: Integration, Experiments and Documentation Owner

Core component: load generation, evaluation and reproducibility

Task	Details
Repo and CI	GitHub repo, structure, branching, README skeleton
Load generator	Background UDP traffic (iperf or custom) with adjustable intensity
Experiment automation	Scripts to start topology, controller, server, clients and load, then collect results
Socket–SDN integration	End-to-end runs, verifying that application events and network behavior interact
Analysis	Graphs (RTT, jitter, sync error), tables, baseline vs SDN comparison
Challenging scenario	Design and run the congestion/loss/failure scenario (**E5 and E6**)
Shared share	Final report, demo script, test-case document

Workload balance

Area	Role 1	Role 2	Role 3
Core build	Socket app	Mininet + SDN	Load gen + automation
Experiments	E1, E2	E3, E4	E5, E6
Documentation	Socket layer	SDN layer	Final report + README
Reviews	Reviews Role 2's code	Reviews Role 3's code	Reviews Role 1's code
Approx. effort	~33%	~33%	~33%

Role 2 has the hardest technical work, while Role 3 has more coordination and documentation. To keep effort balanced, **Role 1 helps Role 2 with OVS queue configuration in the first week**, and **Role 2 helps Role 3 with the final demo and viva preparation**.

Shared responsibilities (everyone)

- Agree on the packet format, DSCP value, ports, IP addressing and CSV log format in the **first meeting**.
- Write the architecture and problem statement together (D1: 2 marks).

- Review each other's code and explain it to the team.
- Run the complete demo end to end at least once each.
- Prepare for the viva: **every member must be able to explain the whole system**, not just their own part.

10. Timeline (about 3 weeks)

Days	Role 1	Role 2	Role 3
1–2	Basic UDP server and client	Install Mininet + Ryu, basic topology	Repo, architecture doc, iperf tests
3–5	T1–T4 math, multi-client threads	Flow rules, DSCP match, OVS queues	Load generator, test scripts
6–7	Error measurement, logging	Connect controller with the topology	**D1 integration and demo**
8–11	Robustness and edge cases	Dynamic SDN logic	Experiment automation
12–15	Experiments E1, E2	Experiments E3, E4	Experiments E5, E6, graphs
16–19	Everyone: documentation, README, results discussion		
20–21	Everyone: demo rehearsal and viva practice		

Adjust the dates to your actual D1 and D2 deadlines.

11. Checklists

D1 checklist

- [ ] Problem statement and architecture diagram written
- [ ] UDP server and client exchange T1–T4 correctly
- [ ] D and θ computed
- [ ] Mininet topology runs
- [ ] Ryu controller installs flow rules
- [ ] Socket app runs inside Mininet with the controller
- [ ] Basic test cases demonstrated

D2 checklist

- [ ] Multiple clients with different skews
- [ ] Priority marking (DSCP) and controller queue assignment
- [ ] Controlled delay and loss
- [ ] Dynamic SDN behavior (congestion-triggered)
- [ ] Challenging scenario demonstrated
- [ ] Experiments E1–E6 with graphs and baseline comparison
- [ ] README with reproduction steps, test cases, full documentation

- [ ] GitHub repo final
- [ ] Viva preparation done by all three members

12. Viva Preparation: Questions to Expect

Why UDP instead of TCP for time synchronization?

Explain T1, T2, T3, T4 and derive the offset formula.

What happens if forward and return delays are different?

How do you simulate a wrong clock in Mininet?

What is DSCP/`IP_TOS`, and how does the switch use it?

How does your controller detect congestion and react?

What is the difference between your baseline and your SDN approach?

What did your graphs show, and why?

What are the limitations of your system?

Explain a piece of your teammate's code.

Tip: keep this file in the repo under `docs/` and update it as the project evolves.